

CMU-CS 462: Software Measurement and Analysis

Spring 2025-2026

Software Reliability Models & Metrics

Nguyen Duc Man
E: mannd@duytan.edu.vn
T: 0904235945

12/01/2026

1

Contents

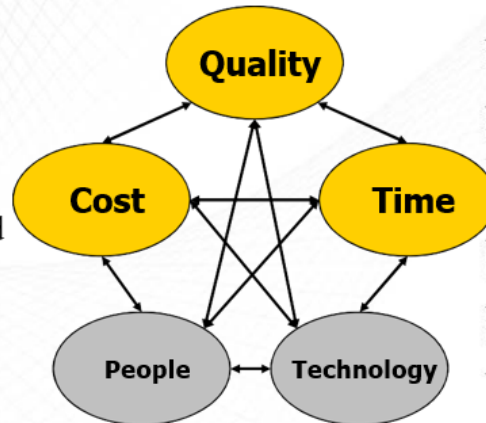
1. Reliability concepts and definitions
2. Software reliability models and metrics
3. Fundamentals of software reliability engineering (SRE)
4. Reliability management models



2

What Affects Software?

- **Time:**
 - Meeting the project deadline.
 - Reaching the market at the right time.
- **Cost:**
 - Meeting the anticipated project costs.
- **Quality:**
 - Working fine for the designated period on the designated system.



3

What Affects Software Quality?

- **Delivery Date:**
 - Meeting the project deadline.
 - Reaching the market at the right time.
- **Cost:**
 - Meeting the anticipated project costs.
- **Performance:**
 - Working fine for the designated period on the designated system, i.e., reliability, availability, etc.

4

Failure, Failure Intensity & Availability

- **Failure:** Any departure of system behavior in execution from user needs or expectations.
- **Failure intensity:** the number of failures per natural unit or time unit. Failure intensity is way of expressing reliability.
- **Availability:** The probability at any given time that a system or a capability of a system functions satisfactorily in a specified environment.

$$Availability = \frac{Uptime}{Uptime + Downtime}$$

5

Reliability (Availability)

Availability %	Downtime per year ^[note 1]	Downtime per quarter	Downtime per month	Downtime per week	Downtime per day
90% ("one nine")	36.53 days	9.13 days	73.05 hours	16.80 hours	2.40 hours
95% ("one and a half nines")	18.26 days	4.56 days	36.53 hours	8.40 hours	1.20 hours
97%	10.96 days	2.74 days	21.92 hours	5.04 hours	43.20 minutes
98%	7.31 days	43.86 hours	14.61 hours	3.36 hours	28.80 minutes
99% ("two nines")	3.65 days	21.9 hours	7.31 hours	1.68 hours	14.40 minutes
99.5% ("two and a half nines")	1.83 days	10.98 hours	3.65 hours	50.40 minutes	7.20 minutes
99.8%	17.53 hours	4.38 hours	87.66 minutes	20.16 minutes	2.88 minutes
99.9% ("three nines")	8.77 hours	2.19 hours	43.83 minutes	10.08 minutes	1.44 minutes
99.95% ("three and a half nines")	4.38 hours	65.7 minutes	21.92 minutes	5.04 minutes	43.20 seconds
99.99% ("four nines")	52.60 minutes	13.15 minutes	4.38 minutes	1.01 minutes	8.64 seconds
99.995% ("four and a half nines")	26.30 minutes	6.57 minutes	2.19 minutes	30.24 seconds	4.32 seconds
99.999% ("five nines")	5.26 minutes	1.31 minutes	26.30 seconds	6.05 seconds	864.00 milliseconds
99.9999% ("six nines")	31.56 seconds	7.89 seconds	2.63 seconds	604.80 milliseconds	86.40 milliseconds
99.99999% ("seven nines")	3.16 seconds	0.79 seconds	262.98 milliseconds	60.48 milliseconds	8.64 milliseconds
99.999999% ("eight nines")	315.58 milliseconds	78.89 milliseconds	26.30 milliseconds	6.05 milliseconds	864.00 microseconds
99.9999999% ("nine nines")	31.56 milliseconds	7.89 milliseconds	2.63 milliseconds	604.80 microseconds	86.40 microseconds

6

Error, Fault and Failure



- An **error** is a human action that results in software containing a fault.
- A **fault** (bug) is a cause for either a failure of the program or an internal error (e.g., an incorrect state, incorrect timing).
- Among the 3 factors only **failure** is observable.

7

Reliability & Reliability Engg

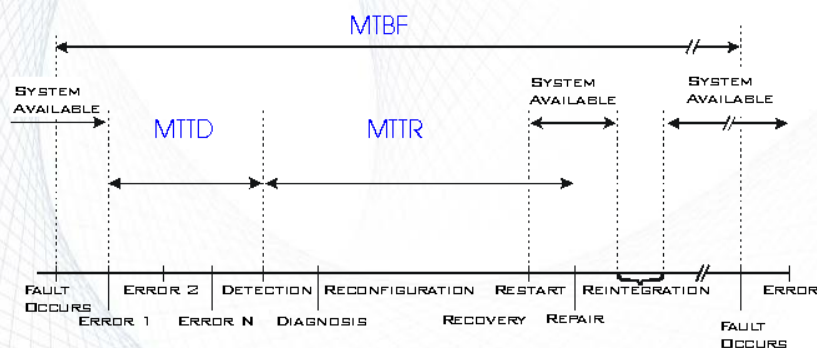
- **Reliability:** The probability that a system or a capability of a system functions without failure for a specified time in a specified environment.
- **Reliability Engineering:** Engineering of reliability in software products.
- **Reliability Engineering's goal:** Developing software to reach the *market*
 - With “minimum” development time
 - With “minimum” development cost
 - With “maximum” reliability

8

Reliability Metrics

- **Mean time to failure (MTTF):** Usually calculated by dividing the total operating time of the units tested by the total number of failures encountered (assuming that the failure rate is constant).
- Example:
 - MTTF for Windows 2000 Professional is 2893 hours or 72 forty-hour workweeks.
 - MTTF for Windows NT Workstation is 919 hours or 23 workweeks.
 - MTTF for Windows 98 is 216 hours or 5 workweeks.
- **Mean time to repair (MTTR):** mean time to repair a (software) component.
- **Mean time between failures (MTBF):**
- $MTBF = MTTF + MTTR$

9



A scenario for on-line detection and off-line repair. The measures - MTBF, MTDD, and MTTR are the average times to failure, to detection, and to repair.

10

Reliability Metrics: Availability

- Software System Availability (A):

$$A(t) = \frac{1}{1 + t_m \lambda(t)} \quad \text{or} \quad \lambda(t) = \frac{1 - A(t)}{t_m A(t)}$$

λ is failure intensity

t_m is downtime per failure

- Another definition of availability:

$$A = \frac{MTTF}{MTTF + MTTR} = \frac{MTTF}{MTBF}$$

- Example:** If a product must be available 99% of time and downtime is 6 min, then λ is about 0.1 failure per hour (1 failure per 10 hours) and MTTF=594 min.

11

Reliability Metrics: Reliability

- Software System Reliability (R):

$$\lambda(t) = \frac{-\ln R(t)}{t} \quad \text{or} \quad \lambda \approx \frac{1 - R(t)}{t} \quad \text{for } R(t) \geq 0.95$$

λ is failure intensity

R is reliability

t is natural unit (time, etc.)

- Example:** for $\lambda=0.001$ or 1 failure for 1000 hours, reliability (R) is around 0.992 for 8 hours of operation.

12

Using Reliability Models

- Suppose that we have developed and tested our software and documented the failures for a certain period of time.
- How do we know that the reliability software is getting better after bug fixes?
- How to measure the increase/decrease of reliability?

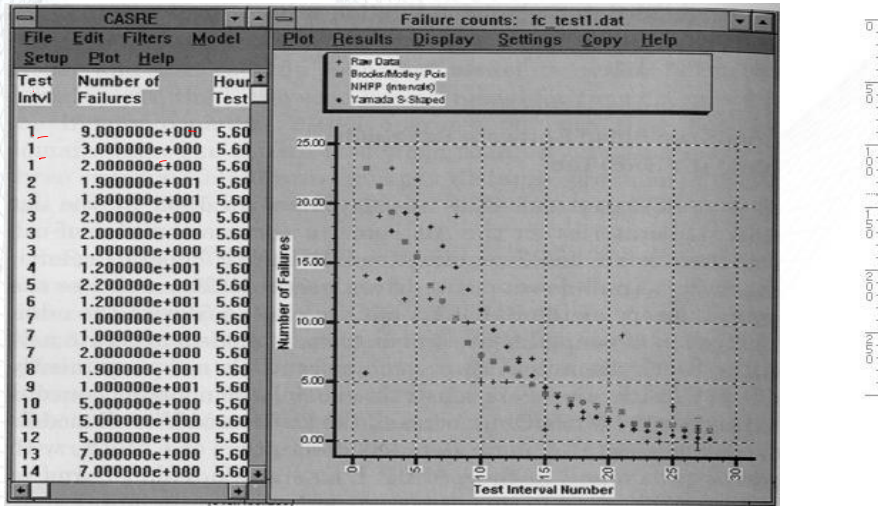
13

SRE Tools: CASRE /1

- Computer-Aided Software Reliability Estimation (CASRE) Tool.
- CASRE is a PC-based tool that was developed in 1993 by the Jet Propulsion Laboratories.
- CASRE requires the WINDOWS operating environment.
- Once the data is entered, CASRE automatically provides the analyst with a raw data plot.
- CASRE provides the analyst with the ability to convert from time-domain data to interval-domain data and vice versa.

14

SRE Tools: CASRE /2



15

Types of Test

- **Certification Test:** Accept or reject (binary decision) an acquired component for a given target failure intensity.
- **Feature Test:** A single execution of an operation with interaction between operations minimized. Sometimes called *Unit test*.
- **Load Test:** Testing with field use data and accounting for interactions.
- **Regression Test:** Feature tests after every build involving significant change, i.e., check whether a bug fix worked.

16

Execute Test

- Allocate test time among the associated systems and types of test (feature, load, regression, etc.).
- Invoke the test cases at random times, choosing operations randomly in accordance with the operational profile.
- Identify failures, along with when they occur.
- Use this information to decide whether release the software or not.

17

Release Criteria

Consider releasing the product when:

1. All acquired components pass certification test
2. Test terminated satisfactorily for all the product variations and components with the λ/λ_F ratios for these variations don't appreciably exceed 0.5 (Confidence factor)

18

What is Software Quality?

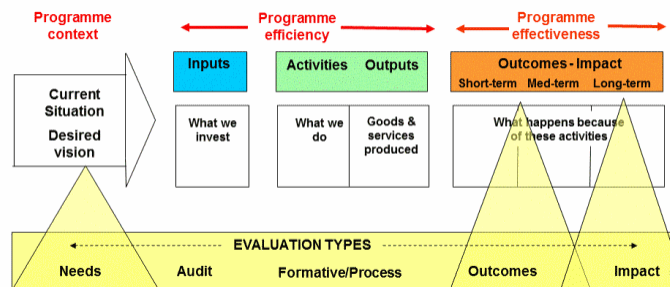
/2

Various viewpoints/perspectives:

- Conformance to customers' requirements
- Requirement, design and development quality
- Process quality vs. end-product quality
 - **Process quality:** higher usability and dependability
 - **End-product quality:** less failure
- Internal vs. external characteristics
 - **Relativity:** advantage over similar products
 - **Conformance:** conformance to standards

19

Selecting Evaluation Techniques



20



Example: Web Server

- Users care about response time (end of response)
- Network capacity is expensive $\Rightarrow$ want high utilization
- Pages delivered per day matters to advertisers
- Also care about error rate (failed & dropped connections)

21



Example: Requirements for Web Server

- 2 seconds from request to first byte, 5 to last
- Handle 25 simultaneous connections, delivering 100 KB/s to each
- 60% mean utilization, with 95% or higher less than 5% of the time
- <1% of connection attempts rejected or dropped

22

Is the Web Server SMART?

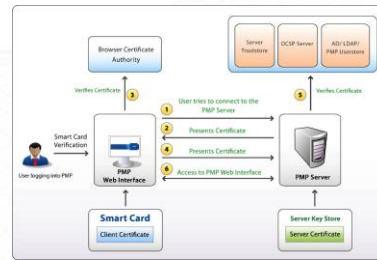
Specific: yes

Measurable: may have trouble with rejected connections

Acceptable: response time and aggregate bandwidth might not be enough

Realizable: requires T3 link; utilization depends on popularity

Thorough? You decide



23

Other Web Server Metrics

Advertisers might want to count pages delivered daily

Error metrics are a big issue:

Availability

Mean Time to Repair

24

Remaining Web Server Issues

Redundancy: response time is closely related to bandwidth, utilization

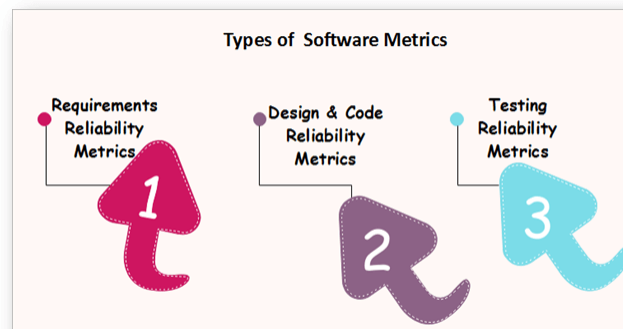
Variability: all measures could vary widely.

Should we specify variability limits for other than utilization?

25

Discuss in Group

Reliability Metrics models



26

References

Standards:

- [IEEE730] IEEE 730-2002: IEEE Standard for Software Quality Assurance Plans, Institute of Electrical and Electronics Engineers, 29-Sep-2002.
- [ISO8402] ISO 8402-1994: Quality management and quality assurance – Vocabulary, 1994.
- [ISO9126] ISO/IEC 9126-2001: Software engineering -- Product quality -- Parts 1-4, ed. 1, Current stage 90.20 JTC 1/SC 7, 2001.

Software Quality:

- [BBK+78] Boehm, B. W., Brown, J. R., Kaspar, H., Lipow, M., McLeod, G., and Merritt, M., Characteristics of Software Quality, North Holland, 1978.
- [Kan02] Stephen H. Kan: Metrics and Models in Software Quality Engineering, 2nd Edition, Addison-Wesley, 2002.
- [MRW77] McCall, J. A., Richards, P. K., and Walters, G. F., "Factors in Software Quality", Nat'l Tech. Information Service, no. Vol. 1, 2 and 3, 1977.

- <https://www.youtube.com/watch?v=28VKi4M8-JI>